Nalanda college of engineering

Internship Report On

PROMPT ENGINEERING

Submitted For The Purpose Of The 2 Weeks

Industrial Training Bachelor of Technology in

“ COMPUTER SCIENCE”

Submitted By

Abhayjeet Kumar

Reg no. : 24105109029

Affiliated To Bihar Engineering University, Patna

CERTIFICATE

This is to certify that the mini-project entitled

“ PROMPT ENGINEERING”

has been completed successfully by: -

Abhayjeet Kumar (24105109029)

Department of Mechanical Engineering,

Nalanda College of Engineering (2022-2026),

Nalanda, Bihar, under the supervision and

guidance of Prof.

MISS PRIYANKA SINHA

during 3rd semester

Prof. PRIYANKA SINHA

C.S.E Department

Nalanda College Of Engineering

Chandi(Nalanda)

Ezyintern

MINISTRY OF CORPORATE AFFAIRS
GOVERNMENT OF INDIA

Ministry of MSME, Govt. of India

CERTIFICATE

OF COMPLETION

This is to certify that

Mr. / Ms. Abhayjeet Kumar

of

Nalanda College of Engineering, Chandi

has successfully completed a **2 Weeks Offline** internship training in

Prompt Engineering

from **24-12-2025** to **02-01-2026 .**

During this internship, **he/she** has learned key concepts in above mentioned domain with practical assignments and project. The student maintained **88%** attendance and secured **95%** marks in the final assessment.

We appreciate **his/her** sincere participation and wish for their best for future opportunities.

AICTE

NATIONAL INTERNSHIP PORTAL

CERTIFIED ISO 9001:2015 COMPANY

EZYINTERN SDP TECHNOLOGY PVT. LTD.

Mr. Raushan Kumar
Founder & CEO

Certificate No. : EZ/2025/13166
Date of Issue : 03/01/2026

(AICTE Approved and ISO Certified Platform)

Online Certificate Verification available on : www.ezyintern.com/certificate-verification/

CANDIDATE DECLARATION

I hereby declare that the work presented in this report entitled **" PROMPT ENGINEERING"** in for the degree of Bachelor of Technology in COMPUTER SCIENCE AND ENGINEERING submitted in the department of Mechanical Engineering, Nalanda College Of Engineering, Chandi, Nalanda is an authentic record of my own work carried out over a period in the month of January 2024 under the supervision of Prof. Priyanka Sinha , of CSE department. The matter embodied in the report has not been submitted for any other degree.

Date:

Place: CHANDI, NALANDA

Abhayjeet Kumar

Reg. No. : 24105109029

This is to certify that the above statement made by the candidate is true to the best of my knowledge.

(supervisor signature)

Prof. Priyanka Sinha

Computer Science Engg. Department

ACKNOWLEDGEMENT

The internship opportunity I had with Engineer Core in association with Academy of Skill Development was a great chance for learning and professional development. Therefore, I consider myself as a very lucky individual as I was provided with an opportunity to be a part of it. I am also grateful for having a chance to meet so many wonderful people and professionals who led me though this internship period.

Bearing in mind previous I am using this opportunity to express my deepest gratitude and special thanks to the MD, who in spite of being extraordinarily busy with her/his duties, took time out to hear, guide and keep me on the correct path and allowing me to carry out my project at their esteemed organization and extending during the training.

Name: Abhayjeet Kumar

Place: NCE CHANDI

Date:

Contents

1. Introduction to Prompt Engineering

Prompt engineering is the discipline of designing and refining inputs to large language models (LLMs) so that they produce accurate, useful, and controllable outputs. A prompt is typically a piece of text (sometimes combined with other modalities such as images) that the model uses as the starting point to predict a sequence of tokens. Because LLMs operate as next-token predictors trained on vast text corpora, the phrasing, structure, and context of the prompt heavily influence the model's behavior.

Anyone can write prompts; you do not need to be a data scientist or machine learning engineer to start working effectively with LLMs. However, crafting high-quality prompts is non-trivial because many factors matter: the specific model used, its training data, configuration parameters, word choice, tone, structure, and the surrounding context. Poorly designed prompts can result in ambiguous or inaccurate responses and reduce the model's ability to produce meaningful output.

Prompt engineering should be treated as an iterative process rather than a one-shot activity. In practice, you experiment with variations of prompts, adjust configurations, observe outputs, and refine based on what works best for a task. Over time, this iterative loop leads to robust prompt templates and patterns that can be reused in applications and systems.

2. How Large Language Models Work

LLMs function as prediction engines. Given a sequence of tokens as input, they predict the probability distribution over possible next tokens, then sample one token according to that distribution. This newly produced token is appended to the sequence, and the process repeats until some stopping criterion (like a maximum length or an end-of-sequence marker) is reached.

The prediction of each next token is shaped by two main elements: the content of the preceding tokens and the patterns the model internalized during training. When you write a prompt, you are essentially setting up the initial sequence from which the LLM will continue, hoping to steer it toward desirable continuations rather than random or irrelevant ones.

LLMs can be used for many tasks simply by changing the prompt:

- Text summarization.
- Information extraction.
- Question answering.
- Text classification.
- Language or code translation.
- Code generation and documentation.
- Reasoning over natural language or code.

While the whitepaper uses Google's Gemini models in Vertex AI as its running example, the principles it describes generalize to other models such as GPT-style models, Claude, or open-source models like Gemma and LLaMA.

3. LLM Output Configuration

Beyond the text of the prompt, model configuration parameters strongly affect the output. Effective prompt engineering requires understanding and tuning these controls:

- Output length (max tokens).
- Sampling controls: temperature, top-K, and top-P (nucleus sampling).

These settings influence the randomness, determinism, verbosity, and cost of model responses.

3.1 Output Length

One of the key settings is the maximum number of tokens to generate in the response. Generating more tokens increases computation, which can raise energy usage, slow responses, and increase costs. Importantly, reducing the max output length does not make the model inherently more concise; it simply forces truncation once the limit is reached.

For tasks that require short answers, you often need to combine a low max token limit with prompt instructions about brevity (e.g., “answer in one sentence”), otherwise the model may begin a longer answer that gets cut off mid-thought. Output length control is particularly critical for techniques like ReAct, where a model might otherwise continue producing unnecessary tokens after the useful part of the response.

3.2 Sampling Controls: Temperature, Top-K, Top-P

LLMs do not directly choose a single token; they compute probabilities over the entire vocabulary and then sample. The three standard controls—temperature, top-K, and top-P—determine how that probability distribution is turned into a single token choice.

Temperature

Temperature controls how “random” the sampling is.

- Low temperature (e.g., 0) corresponds to greedy decoding, where the highest probability token is always selected.

- Higher temperature values allow lower-probability tokens to be chosen more often, leading to more diverse and creative outputs.
- Extremely high temperatures tend to make all tokens roughly equally likely, resulting in highly random, often incoherent text.

In practice:

- Use low temperatures for deterministic tasks requiring a single correct answer (e.g., math problems, strict classification).
- Use moderate to high temperatures for creative tasks where variety is desirable (e.g., brainstorming, storytelling).

Top-K

Top-K sampling restricts the choice of next token to the K most likely tokens according to the model.

- A small K (e.g., 1) makes the model highly deterministic, since only the most probable token is ever used.
- Larger K values allow more possible tokens, making the output more varied and creative.
- If K is set extremely high, such as the vocabulary size, then effectively no tokens are filtered out by K.

Top-P (Nucleus Sampling)

Top-P, or nucleus sampling, chooses the smallest set of tokens whose cumulative probability mass is at most P.

- P near 0 behaves like greedy decoding, considering only the top token.
- P near 1 allows almost all tokens with nonzero probability to be candidates.

Top-K and top-P can be used together or separately, and the best choice is often empirical: experiment to see which combination yields the behavior you want.

3.3 Interaction of Sampling Parameters

When temperature, top-K, top-P, and max tokens are combined, they interact in non-trivial ways. At extreme settings, one parameter can effectively override the others:

- Temperature 0 makes top-K and top-P irrelevant: you always choose the most probable token.
- Top-K = 1 similarly renders temperature and top-P irrelevant, because only one token is considered.
- Top-P = 0 (or very small) restricts sampling to the most probable token, again making other controls moot.
- Very large top-K or top-P = 1 means almost all tokens remain candidates, and temperature dominates randomness.

As a practical starting point, the whitepaper suggests configurations such as:

- “Balanced” creativity: temperature 0.2, top-P 0.95, top-K 30.
- Highly creative: temperature 0.9, top-P 0.99, top-K 40.
- Less creative: temperature 0.1, top-P 0.9, top-K 20.
- Single correct answer tasks: temperature 0.

With more freedom (larger temperature, top-K, top-P, and max tokens), the model is more likely to generate text that drifts from the task or becomes less relevant.

4. Core Prompting Techniques

LLMs are trained to follow natural language instructions, but they are not perfect: vague or underspecified prompts produce poor results. Several well-established prompting patterns help you systematically elicit better behavior.

4.1 Zero-Shot (General Prompting)

Zero-shot prompting gives the model a description of the task and some input, but no explicit labeled examples. This is the simplest kind of prompt—just instructions and content.

For example, a zero-shot prompt might ask the model to classify a movie review as positive, neutral, or negative. In the whitepaper, a simple instruction and a review text are provided, with model configuration set to low temperature (0.1) and default top-K and top-P. Even in this basic setting, the model is often able to infer the task and produce correct labels.

Zero-shot prompting is useful when:

- The task is simple or closely aligned with the model's training distribution.
- You want quick initial experiments before investing in carefully crafted examples.
- There is no easy way to curate high-quality examples.

However, when zero-shot performance is insufficient, you typically move to one-shot or few-shot prompting.

4.2 One-Shot and Few-Shot Prompting

One-shot prompting includes a single example within the prompt that shows the model what the task is and how the response should look. Few-shot prompting extends this idea by providing multiple examples.

Examples serve several purposes:

- They demonstrate desired output structure and formatting.
- They clarify task semantics when instructions alone may be ambiguous.

- They “anchor” the model to a pattern it can follow.

As a rule of thumb, three to five examples are often used for few-shot prompts, though more may be needed for complex tasks and fewer might be required when context length is limited. The whitepaper illustrates this with a few-shot prompt that parses natural language pizza orders into JSON, showing the expected JSON structure before asking the model to handle a new order.

Choosing examples well is critical:

- They should be high quality, correctly labeled, and clearly written.
- They should cover typical cases and edge cases.
- Even small mistakes in examples can confuse the model and lead to undesired output.

4.3 System, Role, and Contextual Prompting

System, role, and contextual prompting all guide the model, but emphasize different aspects.

- System prompting sets the overarching purpose and rules for the model.
- Contextual prompting provides task-specific background information.
- Role prompting assigns an identity or persona to shape tone and style.

System Prompting

A system prompt defines the big picture task and sometimes output format. For example, a system prompt might instruct the model to classify movie reviews and “only return the label in uppercase,” which forces concise, schema-compliant responses. Another example asks for the result in a specific JSON schema, pushing the model to produce structured output while limiting hallucinations.

System prompts are especially useful for:

- Enforcing output structure (JSON, XML, etc.).
- Defining safety constraints (e.g., “be respectful in your answer”).
- Establishing general behavior that many subsequent prompts inherit.

Role Prompting

Role prompting assigns the model a persona such as “travel guide,” “book editor,” or “kindergarten teacher.” This guides not only content but also style, tone, and level of explanation. For instance, the whitepaper shows a prompt where the model acts as a travel guide suggesting three places to visit given a location, and another version that asks for a humorous style.

Defining roles helps:

- Narrow the domain of knowledge and expectations.
- Make outputs more consistent and on-brand.
- Tailor tone for different audiences (formal, humorous, inspirational, etc.).

Contextual Prompting

Contextual prompts feed relevant background information into the model so it can tailor responses to the situation. In one example, the context states that the user is writing for a blog about retro 80s arcade games; the model then suggests article topics that fit that niche.

Contextual prompting is ideal when:

- The task depends on project-specific or domain-specific information.
- You want consistent framing across multiple model calls.
- You need the model to stay within a defined scope.

5. Advanced Reasoning Techniques

Beyond basic prompting styles, several techniques explicitly target reasoning and planning. These methods are especially helpful for complex tasks, multi-step problems, and scenarios where naive prompting fails.

5.1 Step-Back Prompting

Step-back prompting has the model first answer a higher-level, more general question before tackling the specific problem. The general answer is then incorporated into a subsequent prompt to help the model reason better about the original task.

For instance, instead of directly asking for a storyline for a first-person shooter game level, the prompt might first ask for “five fictional key settings that contribute to a challenging and engaging level.” Once those settings are listed, a second prompt uses one of them as context to generate a richer, more creative storyline.

Benefits of step-back prompting include:

- Activating broader background knowledge before honing in on details.
- Encouraging higher-level reasoning and abstraction.
- Mitigating biases by focusing initially on general principles rather than narrow specifics.

5.2 Chain of Thought (CoT) Prompting

Chain of Thought prompting explicitly asks the model to reason through intermediate steps before giving an answer. This can dramatically improve performance on tasks such as arithmetic word problems or logical reasoning, where naive answers are often wrong.

The whitepaper demonstrates this with a simple age puzzle: without CoT, the model returns an obviously incorrect age. When prompted with “Let’s think step by step,” the model breaks down the scenario into numbered steps, computing each element and arriving at the correct result.

Advantages of CoT:

- Often large gains in reasoning accuracy with low engineering effort.
- Better interpretability: you can inspect the model's logic for errors.
- Improved robustness across model versions compared to prompts that only ask for final answers.

Trade-offs:

- CoT increases the number of output tokens, which raises cost and latency.
- The model can still produce flawed reasoning chains; you must verify outputs.

CoT can be combined with one-shot or few-shot prompting, where the prompt includes an example question plus a step-by-step solution, followed by a new question for the model to solve using the same approach.

5.3 Self-Consistency

Self-consistency builds on CoT by sampling multiple reasoning paths and then selecting the most common final answer. Instead of relying on a single chain, the model is prompted several times with high temperature to generate diverse chains of thought; then you extract answers and perform majority voting.

The technique has three main steps:

1. Generate diverse reasoning chains for the same prompt by sampling multiple times.
2. Extract the final answer from each response.
3. Choose the answer that appears most frequently.

This gives a pseudo-probabilistic sense of the most reliable answer and tends to improve accuracy on reasoning tasks. However, it is more expensive because it requires multiple calls per question. The whitepaper illustrates this with an email classification example where multiple chains consider the same email and converge on the classification "IMPORTANT."

5.4 Tree of Thoughts (ToT)

Tree of Thoughts generalizes CoT by exploring multiple reasoning branches in a tree rather than a single linear chain. Each node represents a “thought,” typically a coherent language sequence, and the model explores branches to search for better solutions.

This approach is well-suited for tasks that require exploration and backtracking, such as combinatorial puzzles or complex planning problems. ToT can be implemented by repeatedly prompting the model to expand or evaluate candidate thoughts and using search algorithms (like breadth-first or depth-first) over the tree.

5.5 ReAct (Reason and Act)

ReAct combines natural language reasoning with external actions, such as calling search APIs or code execution tools. Instead of only generating a chain of thoughts, the model alternates between “Thought” and “Action” steps:

- Thoughts: verbal reasoning about the task.
- Actions: calling external tools (e.g., web search) and then incorporating observations into further reasoning.

The whitepaper’s example uses LangChain with Vertex AI and a web search API to answer how many children the members of the band Metallica have. The agent performs a sequence of searches—one for each band member—and accumulates observations before calculating the total.

ReAct is powerful for:

- Complex questions that require retrieving and aggregating external information.
- Tasks where you want the model to decide dynamically which tools to use.
- Building agent-like systems that reason, act, observe, and refine in a loop.

6. Automatic Prompt Engineering (APE)

Automatic Prompt Engineering aims to automate parts of the prompt design process by having models help generate and evaluate prompts. Instead of manually crafting many candidate prompts, you can:

1. Ask a model to propose multiple prompt variants for a task.
2. Score and evaluate those candidates using metrics such as BLEU or ROUGE or task-specific evaluation.
3. Select and refine the best-performing prompts.

The whitepaper shows a simple example: generating ten different phrasings of the same merchandise T-shirt order (“One Metallica t-shirt size S”) for training a chatbot. The model outputs a variety of semantically equivalent instructions, which can then be scored and filtered.

APE is especially useful when:

- You need diverse training or evaluation instructions for a system.
- Human prompt design is time-consuming or repetitive.
- You want to systematically explore the space of possible task instructions.

7. Code-Focused Prompting

Although the core model is text-based, it can handle code tasks very effectively. The whitepaper highlights several patterns for using LLMs as coding assistants.

7.1 Writing Code

You can prompt the model to write code snippets in languages like Bash or Python. In one example, the prompt asks for a Bash script that requests a folder name and then renames all files inside by prepending “draft_” to each filename. The model produces a well-structured, commented script that works correctly when tested.

Key points when prompting for code:

- Clearly specify the language (e.g., “write a Bash script”).
- Describe the input/output behavior step by step.
- Set temperature low for more deterministic, reproducible code.

7.2 Explaining Code

LLMs can also explain existing code. By pasting a script and asking “Explain this code,” you can receive a structured explanation that walks through each part of the script, including input handling, logic, and side effects. This is helpful for onboarding to a new codebase or understanding teammates’ work.

7.3 Translating Code

Another pattern is translating code from one language to another, such as Bash to Python. Given a Bash script, the model can generate a Python equivalent that preserves functionality while adapting to idiomatic constructs. The whitepaper demonstrates translation of the file renaming script into Python, then suggests testing and iterative refinement.

7.4 Debugging and Reviewing Code

LLMs can assist with debugging by analyzing error messages and flawed code. In the whitepaper, a Python script contains a bug (use of a nonexistent toUpperCase function), resulting in a NameError. A prompt that includes the error trace and the code asks the model to debug and improve it.

The model identifies the error, replaces toUpperCase with the correct prefix.upper() method, and suggests additional robustness improvements like error handling and preserving file extensions.

When using LLMs for debugging:

- Always include the error message and stack trace when available.
- Provide the full relevant code section.
- Review the model's suggestions carefully; treat them as proposals, not guaranteed fixes.

8. Multimodal Prompting

Multimodal prompting refers to using multiple input formats—such as text, images, audio, and code—rather than just text. While the whitepaper primarily focuses on text and code, it notes that multimodal models can accept combined inputs to enrich context and improve performance on tasks like image analysis, document understanding, and more.

In practice:

- You might provide an image plus a textual question about it.
- You can combine screenshots of code, log snippets, and instructions.
- You can use documents plus guiding text to extract or summarize information.

The underlying prompt engineering principles still apply: clarity, specificity, and examples remain important even when additional modalities are involved.

9. Best Practices for Prompt Engineering

The whitepaper concludes with a set of best practices distilled from experience. These guidelines help you move from ad-hoc prompting toward a disciplined engineering workflow.

9.1 Provide Examples

Supplying one-shot or few-shot examples is one of the most effective ways to improve prompt performance. Examples serve as reference outputs, teaching the model the desired structure, style, and level of detail. They are particularly important when you want consistent formatting or when the task is complex and instructions alone might be ambiguous.

9.2 Design with Simplicity

Prompts should be clear, concise, and easy to understand both for you and the model. If a prompt is confusing or overloaded with unnecessary details, the model is more likely to misinterpret it. The whitepaper demonstrates rewriting verbose, conversational prompts into concise, action-oriented instructions (e.g., “Act as a travel guide…Describe great places to visit in New York Manhattan with a 3-year-old”).

Using strong action verbs (act, analyze, classify, compare, generate, summarize, translate, etc.) helps convey intent directly.

9.3 Be Specific About the Desired Output

Vague prompts tend to yield vague or unfocused outputs. You should specify:

- Length (e.g., number of paragraphs or sentences).
- Style and tone (conversational, formal, humorous).
- Content constraints (which aspects to include or emphasize).

For instance, “Generate a 3-paragraph blog post about the top 5 video game consoles, in a conversational style” is more effective than “Generate a blog post about video game consoles.”

9.4 Prefer Instructions Over Constraints

Instructions tell the model what to do; constraints primarily tell it what not to do. Research and practice suggest that positive, explicit instructions are generally more effective than long lists of prohibitions.

Constraints are still important—especially for safety, bias mitigation, or strict formatting—but starting with clear instructions and adding constraints only where necessary tends to work better.

9.5 Control Max Token Length

You can manage output length either through configuration (max tokens) or through explicit length instructions in the prompt. A combination of both is often best: instruct the model about length and enforce an upper bound via configuration to control cost and latency.

9.6 Use Variables in Prompts

To make prompts reusable and dynamic, use placeholders or variables for changing parts such as city names, product names, or user attributes. For example, a prompt might say “Tell me a fact about the city: {city},” and your application fills in {city} at runtime. This reduces duplication and helps integrate prompts into software systems.

9.7 Experiment with Input Formats and Writing Styles

Different phrasing, formats (questions vs. commands), and styles can produce noticeably different outputs. For example, describing a game console can be framed as a question, a declarative statement, or an instruction, each leading to different responses. Systematically experimenting with these variations helps you discover what works best for a task.

9.8 Mix Classes in Few-Shot Classification

For classification tasks, ensure that your few-shot examples mix up the order of classes and showcase multiple labels. Otherwise, the model may overfit to a fixed pattern (e.g., always returning the first label shown). Mixing classes helps the model learn to attend to content rather than positional biases.

9.9 Adapt Prompts to Model Updates

Models, architectures, and training data evolve. When you switch to a newer version or a different model, you should:

- Re-test existing prompts.
- Adjust instructions and examples to exploit new capabilities.
- Use tools (like Vertex AI Studio) to store and compare prompt variants.

Prompt engineering is not “set and forget”; it requires maintenance as models change.

9.10 Experiment with Output Formats

For tasks like extraction, parsing, ranking, and classification, structured formats such as JSON or XML often outperform plain text because they are easier to parse and validate programmatically. Asking for JSON also helps constrain the model and reduce hallucinations by forcing adherence to a schema. The earlier movie review classification example illustrates how specifying a JSON schema in the prompt yields structured, ready-to-use output.

9.11 Collaborate with Other Prompt Engineers

When prompt design is difficult, involve multiple people and compare their attempts. Different phrasing and perspectives can produce significantly different performance, even when everyone follows best practices. Sharing examples, failures, and successes accelerates learning and convergence on robust templates.

9.12 Chain-of-Thought Best Practices

For CoT, you should:

- Place the final answer after the reasoning so the model’s intermediate steps can guide the final token predictions.
- Set temperature to 0, because reasoning tasks usually have a single correct answer and benefit from deterministic decoding.
- Make sure you can easily extract the final answer from the response, separate from the reasoning.

These practices make CoT usable in automated systems and enable self-consistency methods to work reliably.

9.13 Document Prompt Attempts

Documentation is critical. Prompt outputs can vary across models, settings, and versions, and even identical prompts can produce slightly different wordings due to tie-breaking in token selection. The whitepaper recommends using a structured template (e.g., a spreadsheet) capturing:

- Prompt name and version.
- Goal.
- Model and version.
- Temperature, token limit, top-K, top-P.
- Full prompt text.
- Example outputs.
- Evaluation notes (OK/NOT OK/SOMETIMES OK).

If you use a tool like Vertex AI Studio, you can also store prompts there and record links in your documentation. For retrieval-augmented systems, you should additionally capture RAG-related settings like queries, chunk sizes, and retrieved context, since they affect what ends up in the prompt.

10. The Prompt Engineering Workflow

Putting all of this together, prompt engineering is best approached as a disciplined, iterative workflow.

Typical steps:

1. Define the task and success criteria.
2. Choose a model and initial configuration.
3. Draft a simple prompt (often zero-shot).
4. Add examples (one-shot/few-shot) and system/role/context prompts as needed.
5. Tune sampling parameters (temperature, top-K, top-P) and max tokens.
6. Introduce advanced techniques (step-back, CoT, self-consistency, ToT, ReAct) for complex tasks.
7. Document results, including both successful and failed variants.
8. Integrate finalized prompts into code, preferably in dedicated prompt files separated from logic.
9. Add tests and evaluation procedures to monitor performance over time.
10. Revisit prompts when models or requirements change.

This process recognizes that prompt engineering is not just about writing a clever instruction once, but about ongoing experimentation, measurement, and refinement.